

HREConnect (Foundation)

Employee & Leave Management System

Capstone Project for Certification | .NET Edition

Contents

Purpose

1) About Us

2) Problem Statement

3) Roles

4) In-Scope Features

5) User Flows (Happy Path)

7) Architecture

8) Tech Stack

9) Data Model

10) API Outline

11) NFRs & SLAs

12) Testing

13) Engineering PoV

14) Delivery Plan

15) Sample Acceptance Criteria

16) Example Seed Data (SQL)

17) Simple Wireframe

18) Sample User Stories

19) Definition of Ready & Done

20) Evaluation Rubric

22) Stretch Goals

23) Getting Started

Purpose

Build a complete Employee & Leave Management System using a React/Angular frontend, ASP.NET Core Web API backend, REST APIs, SQL Server database, authentication, CRUD operations, validations, testing, and deployment fundamentals. The project is intended for engineers who completed full-stack training programs and need a practical enterprise-style capstone project for implementation and certification.

1) About Us

HRConnect focuses on delivering scalable and user-friendly solutions in the domain of employee & leave management. The platform improves HR operational efficiency, reduces manual paperwork, and enables seamless digital experiences for employees, HR managers, and operational teams.

2) Problem Statement

Existing systems related to employee & leave management are often overloaded with unnecessary enterprise complexity and disconnected workflows. Goal: Deliver a simplified but complete enterprise-grade solution demonstrating frontend, backend, database, API integration, authentication, testing, and deployment fundamentals.

3) Roles

1. Guest/User (Employee) – Can apply for leaves and view own records.
2. Admin (HR Manager) – Can manage employees, approve/reject leaves, and run reports.
3. Note: No role-based permissions beyond a simple 'admin' flag.
4. Simple 'Admin Flag' Approach: User Entity includes: bool IsAdmin; When a user registers, IsAdmin defaults to false. Seed one admin user in the database.

4) In-Scope Features

A) Authentication (Basic)

- Register, Login, Logout
- BCrypt password hashing
- Session/token persistence (JWT Bearer)

B) Main Management Module – Employees

- CRUD operations for Employee profiles
- Search and filter employees by name, department, role
- Dashboard showing headcount and department summaries

C) Workflow Module – Leave Requests

- Leave application and cancellation workflows

- Leave status tracking (Pending, Approved, Rejected, Cancelled)
- Leave balance tracking and reporting support

D) Admin Management

- Approve or reject employee leave requests
- View HR reports and operational dashboards

5) User Flows (Happy Path)

1. User (Employee) opens the application.
2. User browses available leave balances and history.
3. User logs in or registers.
4. User submits a leave request.
5. System validates leave balance and stores the request.
6. Confirmation displayed to user.
7. Admin (HR) reviews and approves/rejects leave requests.

7) Architecture

- Frontend: React or Angular
- Backend: ASP.NET Core Web API (.NET 8)
- Database: SQL Server / PostgreSQL
- Authentication: JWT Bearer Token
- Deployment: Localhost

Architecture Layers

- Presentation Layer
- Service Layer
- Repository Layer (with EF Core)
- Database Layer

8) Tech Stack

Frontend

- React + Vite/CRA or Angular
- HTML5, CSS3, JavaScript, TypeScript

Backend

- C# 12, .NET 8, ASP.NET Core Web API
- Entity Framework Core, AutoMapper, FluentValidation

Database

- SQL Server or PostgreSQL
- Entity Framework Core Migrations

Tools & Practices

- Git & GitHub
- Swagger / OpenAPI
- Postman
- Visual Studio 2022 / VS Code

9) Data Model

Entities & Relationships

- User (Id, Email, PasswordHash, IsAdmin, FullName)
- Employee (Id, UserId, Department, Designation, JoiningDate)
- LeaveRequest (Id, EmployeeId, LeaveType, StartDate, EndDate, Status, Reason)
- LeaveBalance (Id, EmployeeId, LeaveType, TotalDays, UsedDays)
- Note: Use simplified entity relationships to keep schema manageable at Fundamentals level.

10) API Outline

Authentication APIs

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout

Main APIs

- GET /api/employees
- GET /api/employees/{id}
- POST /api/employees
- PUT /api/employees/{id}
- DELETE /api/employees/{id}

Workflow APIs

- POST /api/leaves
- GET /api/leaves/mine
- PUT /api/leaves/{id}/status

Error Format (Consistent)

```
{ "timestamp": "...", "path": "...", "error": "...", "message": "..." }
```

11) NFRs & SLAs

Performance:

- Local response time < 1s for API calls.
- Page load < 5s on localhost.

Availability:

- Best effort (developer machine).

Security:

- BCrypt password hashing.
- Basic validations and checks.

Error Handling:

- Structured API errors.
- User-friendly validation messages.

12) Testing

- Backend Unit Tests using xUnit + Moq.
- API Testing using Postman or Swagger.
- Validation testing for workflows.
- Target backend coverage $\geq 60\%$.

13) Engineering PoV

- Follow SOLID principles.
- Use layered architecture.
- Maintain Git branching strategy.
- APIs should be reusable and scalable.

14) Delivery Plan

- Sprint 1: Setup & Core Module
- Sprint 2: Authentication & Workflows
- Sprint 3: Admin, Testing & Documentation

15) Sample Acceptance Criteria

- Users should register and log in successfully.
- CRUD operations for employee profiles should work correctly.
- Employees should submit leave requests and view approval status.
- APIs should return structured responses.

16) Example Seed Data (SQL)

```
INSERT INTO Users(Id, Email, PasswordHash, IsAdmin, FullName)
VALUES ('1', 'admin@example.com', '', 1, 'HR Admin');

INSERT INTO Employees(Id, UserId, Department, Designation)
VALUES ('1', '1', 'Engineering', 'HR Manager');
```

17) Simple Wireframe

- Login / Register Page
- Employee Dashboard
- Leave Request Workflow Page
- Admin – Employee & Leave Approval Management Page

18) Sample User Stories

- As an employee, I want to apply for leave and check my leave balance.
- As an employee, I want to view the status of my submitted leave requests.
- As an admin, I want to manage employee profiles and department records.
- As an admin, I want to approve or reject leave requests.

19) Definition of Ready (DoR) & Done (DoD)

DoR

- Acceptance criteria available.
- API and DB changes identified.

DoD

- Code compiled successfully.
- Manual testing completed.
- Peer review completed.

20) Evaluation Rubric (Aligned to Foundation Skills)

Area	What We Check	Weight
Frontend	Routing, state handling	25%
Backend	REST APIs, validation	25%
Common Skills	C#, SQL, Git	20%
Testing	xUnit/Moq	15%
Generative AI	Copilot/ChatGPT usage	10%
Documentation	README and setup	5%

22) Stretch Goals (Optional, If Time Permits)

- Analytics dashboard with leave utilisation trends
- Search and filter employees by department
- Email notification on leave approval/rejection
- Export leave reports as Excel

23) Getting Started (Suggested Commands)

Backend

```
dotnet new webapi -n HRConnect.API
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet ef migrations add InitialCreate
dotnet ef database update
dotnet run
```

Frontend

```
npm create vite@latest hrconnect-ui -- --template react-ts
cd hrconnect-ui && npm install && npm run dev
```

Run Order

- 1. Start SQL Server / PostgreSQL
- 2. Run dotnet ef database update
- 3. Run ASP.NET Core API (dotnet run)
- 4. Seed initial data
- 5. Run frontend (npm run dev)